[TODO: final title after results]

Arshitha S, Charithra H S, Chandrakala Hipparagi
[TODO: affiliation]

***Abstract*—[TODO: write after results are final]**

***Index Terms*—prompt injection, large language models, LLM security, browser extension, text classification, distribution shift**

I. Introduction

[TODO: write after results are final]

II. Related Work

Attacks. Perez and Ribeiro showed goal hijacking and prompt leaking with hand-written prompts [1], and Greshake et al. moved the payload into third-party content that an application retrieves, the indirect setting we target [2]. Later work studied injection against deployed applications [3], separated out jailbreaks [4], hid instructions inside larger benign tasks [5], and searched for injections automatically [6], [7]. Susceptibility differs across model families [8]. Liu et al. formalized attacks and defenses in one benchmark [9], and BIPIA collects indirect injections in emails, tables and other documents [10]; we use its documents and attack sentences in Section V.

Detectors. Llama Guard classifies prompts and responses against a safety taxonomy [11]; Prompt Guard 2 is a small mDeBERTa classifier for injection and jailbreak intent [12]; PromptShield evaluates detectors under deployment conditions [13]; and Attention Tracker detects injection from the attention of the target model itself [14]. InjecGuard measures over-defense, i.e. benign prompts flagged for looking like attacks [15], and fine-tuned detectors can be broken by adaptive attacks [16]. Baseline defenses such as perplexity filtering trade detection for benign-task performance [17]. PromptLocate localizes the injected span instead of classifying the whole input [18]. Defending a production model needs several layers once retrieval and tools are involved [19], and injection reaches LLM judges and tool selection [20], [21] as well as specialised applications such as resume screening [22].

This work. PromptGuard is not a new detector architecture. It places a detector in the user's browser, in front of any hosted chat service, and asks how far detectors trained on one public dataset carry to others. The name is unrelated to Meta's Prompt Guard models [12], which we use as a baseline.

III. System and Threat Model

A. Components

PromptGuard has three parts (Fig. 1). A Chrome extension watches the prompt box on chatgpt.com, chat.openai.com, claude.ai and gemini.google.com. When the user has stopped typing for **??** ms, it sends the text in the box to a REST API and draws a badge next to the box. The API runs one of the detectors of Section IV and returns a label, a confidence, a risk score, the attack patterns that matched and, when no pattern explains a flag, the words that contributed most to it. A web dashboard issues API keys with a per-key rate limit, shows request logs and has a playground for manual tests (Fig. 2). The code is open source, and the API and dashboard can be self-hosted.

[TODO: Fig. 1, to be drawn in draw.io from the spec in paper/FIGURES.md]

Fig. 1. PromptGuard components and data flow.

The detectors use three labels, SAFE, SUSPICIOUS and MALICIOUS, which the extension shows as Safe, Review and Danger. We use the first set throughout. Since the training data has no SUSPICIOUS class of its own (Section IV), our headline metric is binary: SAFE against not SAFE.

B. Threat model

Two different attacks are often both called prompt injection, and a tool in the user's browser helps against only one of them.

Jailbreaks. The user writes the adversarial prompt, to make the model ignore its own rules. The user is the attacker and controls the browser, so a client-side detector is no security control here: it can be switched off. Jailbreaks are part of our training and test data and we report how well they are detected, but we make no protection claim for them.

Indirect injection. The adversarial text comes from a third party, in an email, a web page, a document or a table, and the user pastes it into the chat together with their own request [2]. The user is the victim. The extension sees the pasted text before it reaches the model and can warn the user. This is the case PromptGuard targets. The attacker controls the pasted content, including its wording, but not the extension, the API or the user's own request. Content that the chat service fetches by itself, such as web results, uploaded files or tool outputs, never passes through the prompt box and is out of scope, as are agents with tool access [23]. Section V tests this case with injections embedded in pasted documents.

C. Privacy

While it is enabled and configured, the extension sends the prompt-box text, the API key and the chosen model to the configured endpoint and nowhere else. The API stores a SHA-256 hash of the text, the label, scores, matched patterns, latency, model and key; the text itself is stored only if this is turned on for the key, which it is not by default. An unsalted hash of a short or common prompt can be guessed, so users who need more than this can self-host the API. The extension

keeps the API key in local, unsynced extension storage, asks for host permission only for the hosted API and, when the user saves another endpoint, for that one origin, and contains no remote code or analytics.

IV. Data and Models

A. Training data

We train and select models on the neuralchemy prompt-injection dataset [24] at a pinned revision and use its official splits: 14036 training, 941 validation and 942 test rows, each labelled benign or malicious. To train the SUSPICIOUS class we derive it inside each split, after splitting: 35% of the SAFE rows get an appended manipulation phrase, and a small set of hand-written suspicious prompts is assigned to exactly one split by a hash of its text. Leetspeak variants of 10% of the MALICIOUS rows and benign hard negatives are added the same way. Any derived row whose normalized text also occurs in another split is dropped, and so is any training row that matches an evaluation row; official evaluation rows are never dropped. The processed splits have 16904, 1162 and 1148 rows. Because SUSPICIOUS is synthetic, every headline number is binary and computed on the 942 original test rows with their original labels.

B. Detectors

Classical. TF-IDF features of word 1–3-grams and within-word character 2–5-grams feed a multinomial logistic regression with balanced class weights. Among 10 candidates (logistic regression with four regularization strengths on word or word-plus-character features, a random forest and multinomial naive Bayes), we keep the one with the highest weighted F1 on the validation split: word-plus-character features with $C = 1$ (validation F1 0.9605; the runner-up scores 0.9596). The fit is deterministic, so all seeds give the same model.

Transformers. We fine-tune DistilBERT (`distilbert-base-uncased`, "bert") and ModernBERT-base ("guard") for 3 epochs at learning rate 5×10^{-5} with inputs truncated to 384 tokens, batch sizes 32 and 8, over 5 and 3 seeds.

Risk score. The API reports $R = \min(100, \max(0,\, b + \min(4m, 15) + 20(c - 0.5)))$, where b is 15, 55 or 85 for SAFE, SUSPICIOUS or MALICIOUS, m is the number of matched patterns and c the model's confidence (its largest class probability). The patterns explain a result but never change the label. The API returns the three terms with the score.

Rule floor and escalation. Two rules sit on top of the classical model. A chat-template token (such as `<|im_start|>`) or text hidden entirely in tags raises SAFE to SUSPICIOUS. When the guard model is loaded, a classical SUSPICIOUS verdict with confidence below 0.65 is replaced by the guard model's verdict. We evaluate both rules in Section V.

V. Evaluation

A. Setup

All models are scored as published or as trained, without tuning on any evaluation set. Inputs pass through the same normalization as the API. We report binary accuracy, false-positive rate (FPR) and threshold-free AUROC, with 95% confidence intervals from 1000 bootstrap resamples of the evaluation set; for small groups we give Wilson intervals instead. Paired comparisons on the same rows use an exact McNemar test. Baselines are ProtectAI's DeBERTa-v3 detector [25] and Llama Prompt Guard 2 [12], used as published (their benign and attack labels map to SAFE and MALICIOUS), plus two rule baselines: our regular expressions alone (any match is an attack) and a fixed keyword list. Latency is CPU time per prompt at batch size 1 on an Intel(R) Core(TM) Ultra 7 255H.

TABLE I
In-distribution results on the 942 original test rows (552 attacks, 390 benign). Accuracy and FPR at the API's operating point with bootstrap 95% CIs; AUROC and FPR at a TPR of 0.95 are threshold-free; ms = median CPU time per prompt.

Detector	Accuracy	FPR	AUROC	FPR@.95	ms
Classical	0.959 [0.945, 0.971]	0.044	0.995	0.033	1.3
DistilBERT	0.958 [0.943, 0.969]	0.041	0.995	0.041	14.5
ModernBERT	0.956 [0.944, 0.968]	0.041	0.996	0.028	66.6
ProtectAI	0.916 [0.897, 0.933]	0.026	0.975	0.085	209.3
Prompt Guard 2	?? [??, ??]	??	??	??	??
Regex only	?? [??, ??]	??	–	–	–
Keyword list	?? [??, ??]	??	–	–	–

B. In distribution

On the original test rows (Table I) the three trained models are statistically tied: the classical model and DistilBERT differ by ?? in accuracy (McNemar $p =$??), and the classical model and ModernBERT by ?? ($p =$??). The classical model needs a fraction of the CPU time. ProtectAI's detector is less accurate here (McNemar $p =$?? against the classical model), mainly because it misses attacks: its true-positive rate is 0.875 against 0.960 for the classical model. The regular expressions match only a few attacks; most attacks in this dataset are obfuscated or reworded.

C. Across datasets

[TODO: external results: Table II, Fig. 3, diagnosis; written once the external run finishes]

D. Injections in pasted documents

Table II uses the threat model's case: 50 emails and 100 tables from BIPIA [10], framed with BIPIA's question as a user would paste them, once clean and once with a BIPIA attack sentence at the start, middle or end. Most BIPIA attacks ask for an unrelated task ("Recommend a good book") rather than overriding instructions; a second variant adds "Ignore all previous instructions." in front of the same sentence.

The three models we trained flag nearly every clean document: the FPR is 0.89 for the classical model and 1.00 and 1.00 for the transformers, so their detection rates carry no information. A document on its own is enough; a table of numbers is flagged with its numbers as the top words. ProtectAI and the rule baselines rarely flag clean documents but catch

TABLE II
PASTED DOCUMENTS WITH EMBEDDED INJECTIONS (BIPIA). FPR ON THE 150 CLEAN DOCUMENTS AND DETECTION RATE FOR THE ATTACK SENTENCE AS PUBLISHED (BY POSITION) AND WITH AN OVERRIDE PREFIX. WILSON 95% CIS ARE IN THE ARTIFACT.

	FPR	BIPIA attack at			with
Detector	clean	start	middle	end	override
Classical	0.89	0.87	0.87	0.87	0.99
DistilBERT	1.00	1.00	1.00	1.00	1.00
ModernBERT	1.00	1.00	0.99	0.99	1.00
ProtectAI	0.08	0.08	0.10	0.09	0.52
Prompt Guard 2	??	??	??	??	??
Regex only	0.00	0.00	0.00	0.00	1.00
Keyword list	0.01	0.01	0.01	0.01	1.00

TABLE III
PERTURBATION ROBUSTNESS: SHARE OF ATTACKS DETECTED / SHARE OF BENIGN ROWS FLAGGED (50 EACH). WILSON 95% CIS ARE IN THE ARTIFACT.

Perturbation	Classical	DistilBERT	ModernBERT	ProtectAI	PG2
none	0.96/0.10	0.96/0.04	0.98/0.08	0.88/0.02	??/??
homoglyph	1.00/0.32	1.00/1.00	1.00/0.96	1.00/0.14	??/??
zero-width	0.96/0.14	0.96/0.04	1.00/0.92	1.00/0.80	??/??
leetspeak	1.00/0.94	1.00/1.00	1.00/1.00	0.96/0.16	??/??
spaced	0.96/0.08	1.00/0.74	1.00/0.70	0.94/0.28	??/??
Base64	0.98/0.12	1.00/0.22	0.98/0.10	1.00/1.00	??/??
German	1.00/0.74	1.00/0.82	0.98/0.58	0.86/0.04	??/??
paraphrase	0.94/0.12	0.92/0.04	0.94/0.02	0.70/0.00	??/??

almost none of the plain BIPIA attacks; they catch the override variant, the rules completely and ProtectAI about half the time. Where the attack lies beyond ProtectAI's 512-token input, it was never detected (14 cases). In short, none of the detectors handles the case PromptGuard is meant for: either everything is flagged or only the explicit override phrase is.

E. Perturbations

Table III applies eight perturbations to 50 attack and 50 benign test rows and shows both classes. Attack detection stays near 1 for every trained model, but the benign column shows why: leetspeak, homoglyphs and German translation make the trained models flag benign text too, up to all of it. The perturbation itself has become the signal. ProtectAI flags few perturbed benign rows, with three exceptions: zero-width characters (0.80), spaced letters (0.28) and the Base64 framing (1.00). Under paraphrasing its detection falls from 0.88 to 0.70; we did not verify that every paraphrase still carries its attack. Base64 is decoded by the API's normalization before any model sees it.

F. Decision pipeline and latency

[TODO: rule floor, escalation sweep (threshold chosen on validation), model latency, end-to-end extension latency]

VI. LIMITATIONS

[TODO: limitations]

VII. CONCLUSION

[TODO: conclusion]

[TODO: Fig. 2: annotated dashboard screenshot, paper/fig2_explanation.png]

Fig. 2. Explaining a flag that no pattern matches: label, risk-score terms and the words that pushed the classical model towards its label.

REFERENCES

[1] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS ML Safety Workshop*, 2022, arXiv:2211.09527.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, arXiv:2302.12173.

[3] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, "Prompt injection attack against LLM-integrated applications," arXiv:2306.05499, 2023.

[4] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, K. Wang, and Y. Liu, "Jailbreaking ChatGPT via prompt engineering: An empirical study," arXiv:2305.13860, 2023.

[5] S. Jiang, X. Chen, and R. Tang, "Prompt packer: Deceiving LLMs through compositional instruction with hidden attacks," arXiv:2310.10077, 2023.

[6] C. Zhang, M. Jin, Q. Yu, C. Liu, H. Xue, and X. Jin, "Goal-guided generative prompt injection attack on large language models," in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2024, arXiv:2404.07234.

[7] D. Pasquini, M. Strohmeier, and C. Troncoso, "Neural exec: Learning (and learning from) execution triggers for prompt injection attacks," arXiv:2403.03792, 2024.

[8] V. Benjamin, E. Braca, I. Carter, H. Kanchwala, N. Khojasteh, C. Landow, Y. Luo, C. Ma, A. Magarelli, R. Mirin, A. Moyer, K. Simpson, A. Skawinski, and T. Heverin, "Systematically analyzing prompt injection vulnerabilities in diverse LLM architectures," arXiv:2410.23308, 2024.

[9] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium*, 2024, arXiv:2310.12815.

[10] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proc. ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025, arXiv:2312.14197.

[11] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama guard: LLM-based input-output safeguard for human-AI conversations," arXiv:2312.06674, 2023.

[12] Meta, "Llama prompt guard 2 86m: Model card," Hugging Face model repository `meta-llama/Llama-Prompt-Guard-2-86M`, 2025, llama 4 Community License.

[13] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," in *Proc. 15th ACM Conference on Data and Application Security and Privacy (CODASPY)*, 2025, arXiv:2501.15145.

[14] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, "Attention tracker: Detecting prompt injection attacks in LLMs," arXiv:2411.00348, 2024.

[15] H. Li and X. Liu, "InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models," arXiv:2410.22770, 2024.

[16] N. V. Pandya, A. Labunets, S. Gao, and E. Fernandes, "May I have your attention? Breaking fine-tuning based prompt injection defenses using architecture-aware attacks," arXiv:2507.07417, 2025.

[17] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P. yeh Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, "Baseline defenses for adversarial attacks against aligned language models," arXiv:2309.00614, 2023.

[18] Y. Jia, Y. Liu, Z. Shao, J. Jia, and N. Z. Gong, "PromptLocate: Localizing prompt injection attacks," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2026, to appear; arXiv:2510.12252.

[19] C. Shi, S. Lin, S. Song, J. Hayes, I. Shumailov, I. Yona, J. Pluto, A. Pappu, C. A. Choquette-Choo, M. Nasr, C. Sitawarin, G. Gibson, A. Terzis, and J. Flynn, "Lessons from defending Gemini against indirect prompt injections," arXiv:2505.14534, 2025.

[20] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, "Optimization-based prompt injection attack to LLM-as-a-judge," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024, arXiv:2403.17710.

[21] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in LLM agents," arXiv:2504.19793, 2025.

[22] H. Mu, J. Liu, K. Wan, R. Xing, X. Chen, T. Baldwin, and W. Che, "AI security beyond core domains: Resume screening as a case study of adversarial vulnerabilities in specialized LLM applications," arXiv:2512.20164, 2025.

[23] S. Willison, "The lethal trifecta for AI agents: Private data, untrusted content, and external communication," Blog post, https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/, Jun. 2025.

[24] NeurAlchemy, "Prompt-injection-dataset," Hugging Face dataset `neuralchemy/Prompt-injection-dataset`, revision 7d70432, apache-2.0 per the dataset card.

[25] ProtectAI, "deberta-v3-base-prompt-injection-v2," Hugging Face model `protectai/deberta-v3-base-prompt-injection-v2`.

RESPONSE TO REVIEWERS

We thank all reviewers. The paper was reviewed at three venues (ICDDS 2026 #1316, INDICON 2026 #4970, IEEE ICCD-2026 #82); many comments overlap, so we answer them by theme and then list the remaining points per review. Section, table and figure numbers refer to the revised paper.

A. Main concerns shared by several reviewers

1) **The BERT and guard tiers were not evaluated** (ICDDS R4, R5; INDICON R1, R2.1; ICCD 3.1). *Done.* DistilBERT and ModernBERT are trained over 5 and 3 seeds and evaluated on the same official test split as the classical model, with bootstrap CIs and paired McNemar tests (Table I), on three external datasets (Section V-C), on pasted documents (Table II) and under perturbations (Table III). All sentences about hosting limits and storage quotas are removed.
2) **The 35-prompt test bed was too small** (ICDDS R4; INDICON R1, R2.2; ICCD 3.2). *Done.* The hand-written 35-prompt table is retired. We report the official test split (942 original rows) and three public external sets with AUROC, FPR, FPR at fixed TPR, confusion matrices and 95% CIs; small groups get Wilson intervals and raw counts.
3) **No baselines; compare with Llama Prompt Guard 2** (ICDDS R4; INDICON R2.6; ICCD 3.2, Q3). *Done.* We add Llama Prompt Guard 2-86M and ProtectAI's DeBERTa-v3 detector, both used as published, and regex-only and keyword baselines, on every evaluation set. Q3 asked for the comparison on the 35-prompt benchmark; since that benchmark is retired, the comparison is made on the public sets instead. We note that neither public detector fully discloses its training data, so overlap with the external sets cannot be excluded.
4) **Training data, splits and model selection were not documented; leakage could not be ruled out** (ICDDS R4; INDICON R2.3; ICCD Q1). *Done* (Section IV): dataset and pinned revision, split sizes before and after derivation, how SUSPICIOUS is derived, cross-split de-duplication, the candidate set and the validation score of the selected model, and the transformer hyperparameters. The earlier text wrongly said the classical model was an ensemble chosen by cross-validation; it is one model chosen on the validation split, and the text now says so. Code, data revision and the commit used for the paper are released.
5) **Risk score, Review band and escalation threshold were not defined** (INDICON R2.3, R2.4; ICCD minor). *Done.* The risk-score formula is given exactly as the API computes it (Section IV), and the API now returns its three terms, so every score is decomposable. The 0.65 escalation threshold is evaluated in a sweep with the threshold chosen on the validation split only (Section V-F). **[TODO: one sentence on whether escalation helps, from pipeline_eval.json]**
6) **Inconsistent labels** (ICDDS R4; INDICON R2.4, Figures). *Done.* The paper uses SAFE / SUSPICIOUS / MALICIOUS throughout, maps them once to the UI names (Safe / Review / Danger), and reports binary SAFE-vs-not-SAFE as the headline because SUSPICIOUS is synthetic.
7) **"Real-time" and "lightweight" were not measured; how does debouncing interact with the rate limit?** (ICDDS R4; INDICON R2.7; ICCD 3.5, Q2). *Done.* Table I gives CPU latency per model, and Section V-F gives end-to-end extension latency (last keystroke to badge) and the request rate of scripted typing against the per-key limit. **[TODO: one sentence with the numbers]**
8) **Threat model unclear; jailbreaks vs injection; pasted documents not tested** (ICDDS R4; INDICON R2.8). *Done.* A threat-model subsection separates jailbreaks (the user attacks; no protection claim) from indirect injection in pasted third-party content (the user is the victim; our target), and Table II tests that case with BIPIA documents. The result is negative and reported as such: the models trained on one public dataset flag nearly every clean document.
9) **The 30% false-positive rate makes the tool unusable** (ICDDS R4; INDICON R2.4; ICCD 3.3). *Addressed.* The 30% figure came from 10 hand-written benign prompts. On the official test split the FPR of the classical model is 0.044; on other datasets and on pasted documents it is much higher (Section V-C, Table II). We now state plainly that the detectors do not generalize well enough to block prompts, and we give FPR at fixed TPR so readers can judge operating points.
10) **Adaptive attacks and over-flagging** (INDICON R2.5, R2.9). *Done.* Table III applies eight perturbations to attack *and* benign rows. It confirms the reviewer's suspicion: the perfect scores on encoded or translated attacks came from flagging anything unusual, which the benign column now shows.
11) **Privacy of the tool** (ICDDS R4; INDICON R2.9). *Done.* Section III states what is sent, what is stored (a hash, not the text, by default), the permissions requested, and the self-hosting option.
12) **Prompts flagged with "no patterns listed"** (INDICON R2.5, Figures; ICCD minor). *Done.* When no pattern matches, the API returns the words that contributed most to the label, computed exactly from the classical model's weights; Fig. 2 shows an example with the risk-score terms.
13) **Novelty; name clash with Meta's Prompt Guard** (INDICON R2.10; ICCD originality). *Done.* Section II states that PromptGuard is not a new detector architecture: the contributions are the browser-side deployment and the evaluation of in-distribution versus cross-dataset generalization. It also states that the name is unrelated to Meta's Prompt Guard.

B. Remaining points

- **Typos and broken text** (all venues: "74.3a", "uni-tested", "across across", "OW ASP", "ENABLE HEAVY MODELS"): removed with the rewrite; the paper no longer contains implementation names.
- **Revision artefacts and implementation detail in the text** (INDICON R2; ICCD 3.4): removed ("now implements", "has since migrated", SQLAlchemy, Alembic, JWT, environment variables, storage quotas).
- **References** (ICDDS R4; INDICON R2; ICCD): every entry was checked against arXiv or the publisher. [9] now lists its authors as on arXiv (first author V. Benjamin, last author T. Heverin; the earlier "R. Shen, M. J. Smith" was wrong). We also corrected the author lists of Liu et al. (prompt injection and jailbreaking papers) and Zhang et al. (now cited at ICDM 2024), the year of PromptShield (CODASPY 2025), added arXiv IDs to Attention Tracker, InjecGuard and the resume-screening study, cite Prompt Guard 2 by its model card, mark the remaining blog post as such, and removed the other.
- **Section order and numbering** (not raised, found by us): the misplaced subsection, the Future Work / Limitations order and a sentence without a main verb are fixed.
- **Fig. 1 inconsistent with the text and without the escalation path; Fig. 2 illegible** (INDICON R2, ICCD minor): Fig. 1 is redrawn with the same labels as the text and shows escalation; Fig. 2 is replaced by a large annotated screenshot.

- **Tables and figures looked AI-generated** (INDICON R1): tables are now LaTeX `booktabs` generated from the experiment outputs, and the diagram is drawn by hand.
- **Percentages for tiny categories** (ICCD minor): no table reports a percentage for fewer than five samples; perturbation and document groups have 50 or more rows each.
- **"Fully decomposable" vs unexplained scores** (ICCD minor): reconciled; the score is decomposed into its three terms for every prediction, and unexplained flags now come with contributing words.